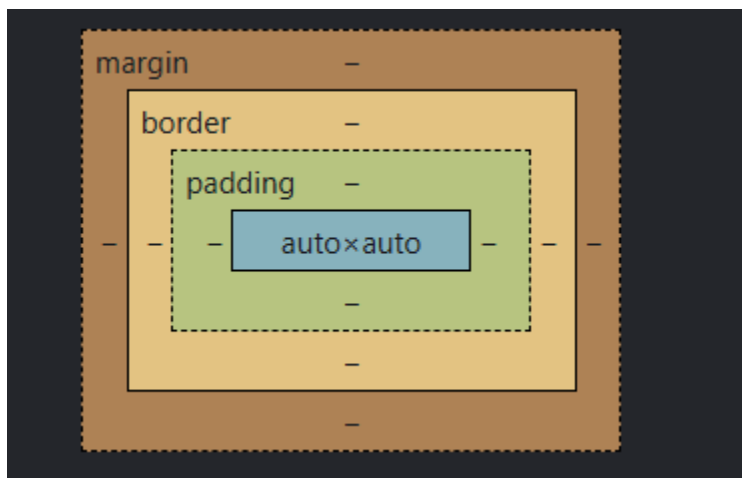


1/14/25

- Fitt's law- easier to move mouse to something that is closer and bigger. $\log(D/S)$
- Steering law- harder to navigate dropdown menu that is long and wide
- People can process visual and audio input at the same time
- Miller's law- 7 +/- 2 chunks can be held in working memory
- Gulf of execution- discrepancy between what a user wants to do and what they can actually do
- Gulf of evaluation- discrepancy between what the user knows/understands about the state of the system and the actual state of the system
- Affordance- what a system lets you do
- Signifier- signals where and how interaction should take place
- Metaphor- way to create a signifier
- Learned association

1/16/25



- CSS selector- select all elements that have a specific tag or...
- # - ID
- . - class
- Selectors can be a little more complicated
 - p, #helloWorld selects all <p> tags and the element with ID "helloWorld"
 - p #helloWorld selects an element with ID "helloWorld" that's inside a <p> tag
 - p#helloWorld selects all <p> tags with ID "helloWorld"
- Position property values
 - static (default)
 - relative- position element relative to its normal/original position
 - top: 10px moves the element 10 pixels down from its original position
 - fixed- element stays in the same position even if the page scrolls
 - right: 0 positions the element 0 pixels away from the right edge of the screen
 - sticky (probably won't use this one often)

- absolute- position element relative to its nearest ancestor that has a specified position.
 - “Specified position” means you explicitly wrote out the element’s position property and assigned it to something besides static.
 - Common use: assign position: relative to the parent element, then assign position: absolute to the child element. Then you can position the child element inside the parent element much like you can position an element with a fixed position inside a webpage.
- With flexboxes (edit position of elements that are children of this container):
 - Choose how items are positioned horizontally using the “justify-content” property
 - Choose how items are positioned vertically using the “align-items” property
 - Choose whether items should be in a row or a column with the “flex-direction” property (default is row, column reverses justify-content and align-items)
 - Specific children elements can override this using the “align-self” property
 - Choose whether items wrap using the “wrap” property (whether they all go on one line or multiple lines)
 - Choose how much space is between lines using the “align-content” property

11/21/25

- JavaScript can be embedded in HTML, but only in the <head> tag or at the end of <body>.
- Alternatively (recommended), you put it in a separate file much like styles.css. Do <script src= “(source here)”></script>.
 - Can be put in <head> or at the end of <body>. If in <head>, it runs before parsing the HTML. If at the end of body (recommended), it runs after parsing the HTML. If you really want to put it in <head> though but make the code run after parsing the HTML, you can write “defer “ before “src”.
- In a button, you can add an “onclick” attribute and set it equal to a line of code (probably a call to a function that was defined in your javascript file) that will run when the button is clicked
- JavaScript selectors
 - document.getElementById(“str”)
 - document.getElementsByClassName(“str”)
- In JavaScript, functions are first-class objects (can be stored in a variable, passed to or returned from a function)
- Functions can be object constructors. Just use “this.propertyName” to assign values to properties of the new object being constructed. Then you can create the object by doing something like “let person = new Person(“mark”)” where Person is the name of the function.

1/23/25

- jQuery
 - \$('h1') selects all h1 tag elements
 - \$('#item') selects the element with ID 'item'

- `$('.items')` selects all elements with class 'items'
- `$(':button')` selects all elements with type 'button'
- `$('[href]')` selects all elements with an href attribute
- `$("[value='Submit']")` selects all elements whose value attribute has value "Submit"
- `$(document)` selects whole document
- `$('#item').html()` gets the element's text
- `$('#item').click(callbackFunc)` causes callbackFunc to be called every time the element is clicked
- `$('#item').css("property", "value")` sets the element's property to the specified value
- `$('#item').append(element)`

1/28/25

- One browser process per window (think: everything above and including the bookmarks bar), one renderer process per tab (think: everything below the bookmark bar)
- Browser process knows user inputs. Once it detects an event, it tells the renderer process that the event occurred, and then the renderer process handles that event.
- AJAX allows you to update parts of a webpage without reloading the whole page.
- There is an event loop that executes code. First it runs all the synchronous code, then it runs all the code in the microtask queue (promises), then it runs all the code in the macrotask queue (everything else, like `setTimeout`, `setInterval`, `ajax`). Before code/functions enter either of these queues, we first wait on the web API to return a response.
 - For example, while we're waiting for a promise to resolve, that promise is just waiting on the web API. But once it's resolved, if there is a callback function that is called once that promise resolves, it would go in the microtask queue.

1/30/25

- Want to make systems **usable and useful** by focusing on users, needs, requirements. It's iterative.
- Four stages of Human-centered design- need finding, iteration, prototyping, testing
- Can try to understand users through observation, surveys, or interviews (either structured, semi-structured, or unstructured).
 - Ask the right people (daily users, manufacturers, experts) the right questions (why do you do that/want this feature?)

2/4/25

- Ask targeted, specific questions- easier to answer than general questions
- Qualitative data is important to help you know your user's needs
- Affinity diagrams:
 - 1. Capture the main idea of each interview. Your (team's) interpretation
 - Each interpretation note should contain one idea, be self-contained, relate to the participant's motivation, and use first-person pronouns (e.g "I").

- 2. Group interviews that are similar to each other into clusters called affinity groups
- 3. Label each affinity group
 - Group hierarchically; data (white) > category (blue) > theme (pink) > insight (green)
 - Pink and green labels should be about an issue or area of concern
 - Make labels based on the data. Don't try to make categories that fit your pre-conceived labels.

2/6/25

- IDEO and Double diamond are two design processes
- IDEO rules:
 - 1. Defer judgement
 - 2. Encourage wild ideas
 - 3. Build on others' ideas
 - 4. Stay focused on topic
 - 5. Conversations take place one at a time
 - 6. Be visual
 - 7. Go for quantity
- Speed dating- discover what users like and dislike
 - Done by showing users storyboards
 - 3-4 panels
 - Being handdrawn is fine
 - Include some riskier/less rational designs
 - Storyboards do not need to focus on *how* the technology works, but more so on how it's used/user experience
 - Goals: need validation and getting user's thoughts on proposed solutions

2/11/25

- A prototype is:
 - A series of screen sketches
 - A storyboard
 - Powerpoint slides
 - A simulation
 - A physical rough model
 - A piece of software with limited functionality
- Purpose: can see what it's like, test out new ideas, get feedback, communicate effectively
- Can be low-fidelity or high-fidelity
- A wireframe is a rough sketch of a user interface
- A wizard-of-oz model uses a human to mock certain functionality
- Figma is the most popular prototyping application
- Prototyping is iterative- you can start with a high or low fidelity prototype, and make more as you go on

2/13/25

- Figma pages- like snapshots of layouts. Think of it like a git commit.
- A frame is like an HTML div
- Constraints let you specify where an element should lie relative to the edges of the element it is inside. Like specifying UI element anchors in Unity.
- Auto layout is like a CSS flexbox.
 - Rectangle select multiple elements and make it an auto layout to group them together.
- A component is like a Unity prefab.
 - Instances can be created by holding the alt key and holding down the mouse and dragging out of the component
 - Can view in Assets tab
 - Don't use them in your designs, only instances of them
- Prototyping lets you define interactions.
 - Go to the prototype tab, hover over an element, click the + button, and drag to another component
 - From there, you can define the interaction much like a JavaScript event
 - Click the present button to test it out
- Variants of components can be made, and interactions can be made between a component and its variants
- Variables are only available with the education or paid plan

2/18/25 was a guest lecture, 2/20/25 was the day of the midterm so there was no lecture.

2/25/25

- Discount usability methods- not user testing, but fast and cheap
 - Cognitive walkthroughs
 - Heuristic evaluations
- Nielsen's 10 usability heuristics
 - Visibility of system status
 - The user should know what's going on inside the system
 - Match between system and real world
 - The design should speak the users' language
 - User control and freedom
 - Should have a way to easily undo a user mistake
 - Consistency and standards
 - Users shouldn't have to worry whether different words/situations/actions mean the same thing
 - Error prevention
 - Prevent the user from making a mistake (ex: github making you confirm when you want to delete a repo, autocorrect)
 - Recognition rather than recall
 - List all options out rather than making the user remember them (ex: autocomplete)

- Flexibility and efficiency of use
 - Shortcuts- hidden from novice users- can speed up interactions
- Aesthetic and minimalist design
 - Should not include irrelevant/rarely needed information
- Help users recognize, diagnose, and recover from errors
 - Error messages should be in plain language, precisely indicate problem, and suggest a solution
- Help and documentation
 - If a user needs this, something is wrong with the interface. But if they need it, give them good help.

3/11/25

- Think-aloud protocol- ask the user to think out loud while using a prototype of your user interface
 - The task you ask them to do should be common and concrete
 - Before starting the think-aloud, introduce your app and explain how to do the think-aloud
 - Tell them to try to not explain things, just verbalize their thoughts
 - Tell them I won't help unless absolutely necessary
 - Describe the task- give written instructions
 - Don't interrupt them while they do this- only ask questions about why did they did certain things after the session is over
 - Remind them to say their thoughts out loud if they haven't spoken in a while
 - Take notes and look for users' strategies and mental models
- Usability finding templates
 - Make each time you view behavior/see evidence of positive or problematic experience
 - Give it unique UFT ID, name, and connect them at the end
 - Make note of frequency (common?), impact (easy to overcome?), persistence (one-time problem?)

3/13/25

- Vue.js - a JavaScript framework for making webpages. Simpler than React.js

Same code in plain JavaScript vs Vue.js

Vue.js

Bind the view with the data, it'll display the value of the 'product' data property in the Vue instance →

Create a Vue application →

Defines the data object for the Vue application. It returns an object that contains the 'product' property →

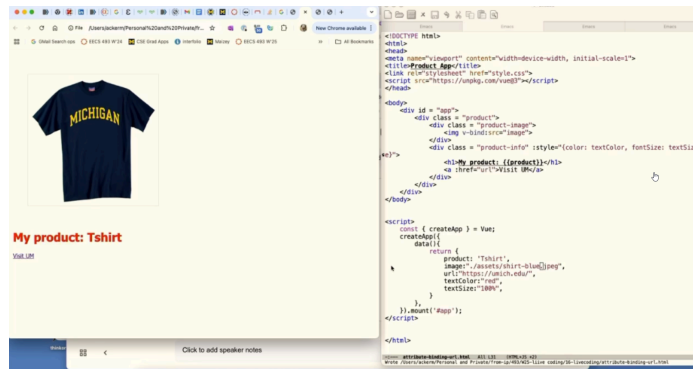
Mount the Vue application to DOM →

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>Product App</title>
5 <script src="https://unpkg.com/vue@3"></script>
6 </head>
7
8 <body>
9   <div id="app">
10     <h2>My Product:</h2>
11     <h1>{{ product }}</h1>
12   </div>
13 </body>
14
15 <script>
16   const { createApp } = Vue;
17
18   createApp({
19     data() {
20       return {
21         product: 'Boots',
22       }
23     }
24   }).mount('#app');
25 </script>
26 </html>

```

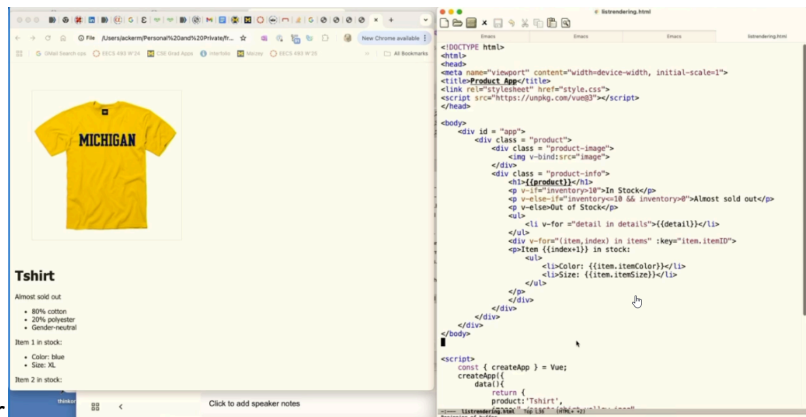
- Basic setup steps:
 - 1. Create app
 - 2. Create data
 - 3. Create methods
 - 4. Connect the Vue instance to an HTML element
- Can put JavaScript expressions in `{{}}`, such as keys in the dictionary that `data()` returns
- Directives coordinate program flow



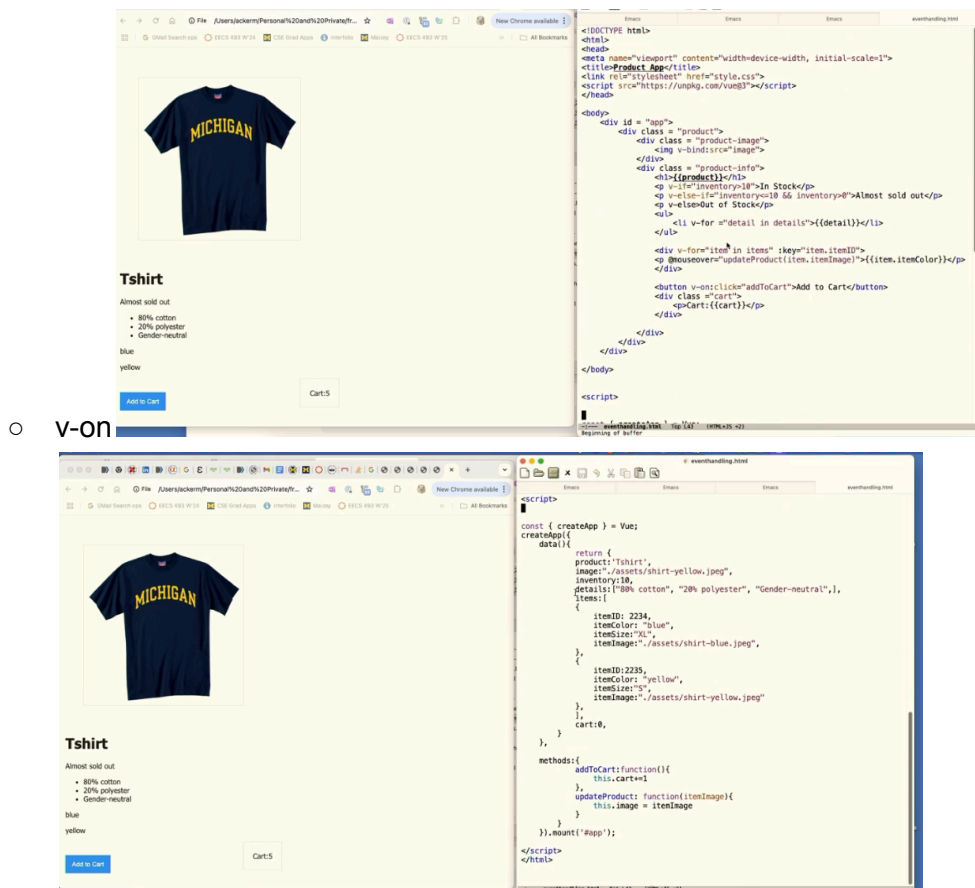
- v-bind
 - Allows you to access Vue data in HTML element attributes (if a string in the attribute value is one of the data keys, it will be replaced with the corresponding value)
 - Can use v-bind: or just :
 - Can bind a class
 - `:class = "{disabledButton: inventory == 0, lowStock: inventory > 5}"`
 - disabledButton and lowStock are class names



- v-if



- v-for



3/18/25

- Bootstrap is responsive- looks good even if you change screen size
- Organizes page into 12 columns
- Classes include:
 - container (margin)
 - container-fluid (no margin)
 - bg-primary, bg-success, bg-danger (set background color)
 - mt-4 (margin top)
 - Text-white
 - p-3 (padding size)
 - nav-X (navigation)
 - btn btn-X (buttons)
 - dropdown-menu
 - dropdown-item
- JSON- string that represents an object that follows strict syntax
- JSON.parse and JSON.stringify
- In JSON, strings need to be in double quotes (in JS, they can be in single quotes)
- JSON cannot contain functions
- If your JavaScript object references another object, JSON includes the referenced object

JSON object references

- JSON doesn't know they are the same object

```
> JSON.stringify(XuOffice)
< '{"name":"Xu Wang","building":{"name":"Beyster Building","address":"2260 Hayward Street","city":"Ann Arbor","state":"Michigan"},"roomNumber":3737}'

> JSON.stringify(MarkOffice)
< '{"name":"Mark Ackerman","building":{"name":"Beyster Building","address":"2260 Hayward Street","city":"Ann Arbor","state":"Michigan"},"roomNumber":3912}'

> JSON.parse(JSON.stringify(XuOffice)).building
< {name: 'Beyster Building', address: '2260 Hayward Street', city: 'Ann Arbor', state: 'Michigan'}

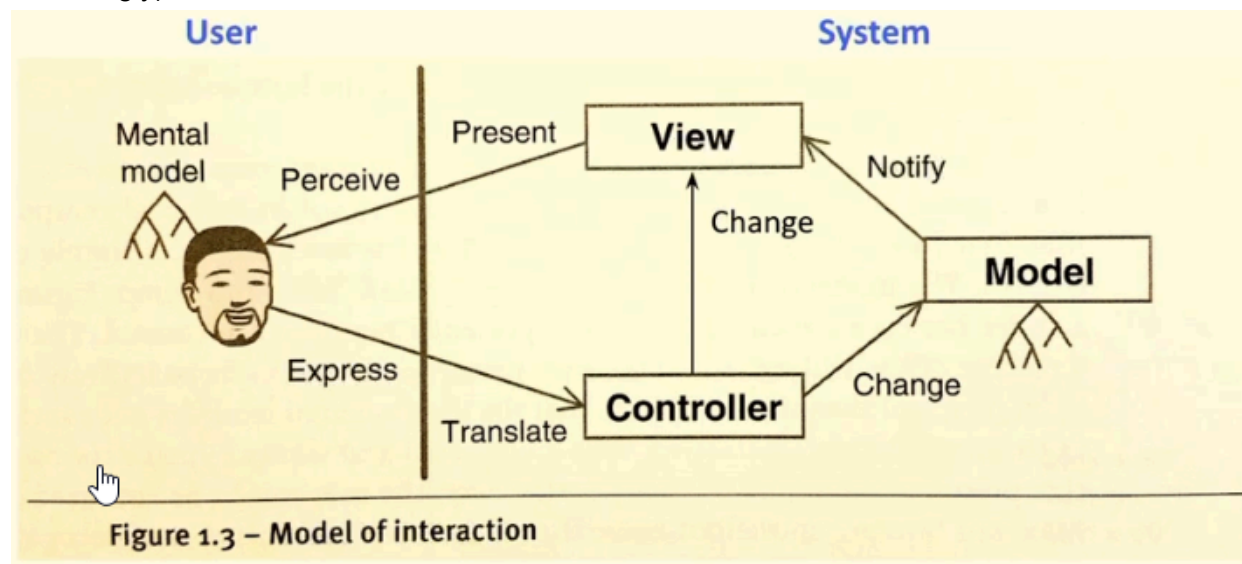
> JSON.parse(JSON.stringify(MarkOffice)).building
< {name: 'Beyster Building', address: '2260 Hayward Street', city: 'Ann Arbor', state: 'Michigan'}

> JSON.parse(JSON.stringify(XuOffice)).building == JSON.parse(JSON.stringify(MarkOffice)).building
< false
```

-
- Get around this by adding ID property, then compare IDs

3/20/25

- Model-View-Controller
 - Model- system state (ex: JSON, SQL database)
 - View- what users see (ex: spreadsheet view, pie chart)
 - Controller- how stuff gets changed (detects input/events, updates stuff accordingly)



-
- View should be separated from domain logic and model so that the same view can be used across many applications
- View and model should be decoupled

- There are variants of MVC such as MVVM, but they all have separation of concerns, modularity, and reusability
- Tool- software program/utility that helps solve a problem.
 - Ex: VS Code
 - Usually language-agnostic.
- Library- collection of functions that help solve a problem.
 - Ex: jQuery
 - Usually language-specific
- Framework- scaffolds the creation of a tool or application. Has initial components and definable slots that developers can use to accomplish a task
 - Ex: Vue, Bootstrap, React
 - Usually language-specific
- Inversion of control. Library- you get more control, Framework- it gets more control

3/25/25

- Lecture reviews design and development process of a user interface that uses AI to assist feedback provision
- AI gives ECON 101 graders potential feedback they can give to students for essays they are grading
- Users (graders) can see each rubric item on screen. Highlighted green if AI thinks student met that criteria, otherwise red, and additional feedback.
- 4 types of changes made
 - Changes of UI elements
 - Minimize cursor travel
 - Reduce unnecessary buttons (make existing UI elements interactive)
 - Visualizations for discoverability and understandability
 - Changes based on users' mental models
 - Changes for responsible use of AI
 - Changes based on technical feasibilities

3/27/25

- About 1 billion people (15%) have a disability
- Disability- mismatched interaction between someone and their context
- Accessibility- tools that help people navigate mismatched interactions
- Inclusive design- framework that helps design more accessible products
- Assistive technology- improve ability of disabled people
 - Screen readers, screen adjustment, speech input, keyboarding, etc.
- Universal design- one design works for everybody. Seven principles
 - Equitable use
 - Flexibility in use
 - Simple and intuitive use
 - Perceptible information
 - Tolerance and error
 - Low physical effort

- Size and space for approach and use
- Ability-based design- technology that adapts based on ability of user
- Accessible design- people with disabilities explicitly considered when designing

4/1/25

- AI has uncertain and unpredictable output
- AI lack common sense/abstract concepts to refine interpretation
- Machine learning is dynamic and solutions are not always motivated by real problems
- UI can try to correct for mistakes in AI output, but training an AI to minimize error would be better
- Guidelines for AI-Human interaction
 - Initially
 - During interaction
 - When wrong
 - Over time
- With LLMs, can have narrow gulf of execution but wide gulf of evaluation
- Gulf of envisioning- gulf between what a user envisions/wants and their ability to express it
- Design patterns
 - Visually track prompts and outputs
 - Suggest ideas for prompting
 - Provide multiple outputs
 - Make the output explainable
 - Use domain-specific prompting strategies
 - Allow manual control of output

4/3/25

- Social computing has:
 - Roles
 - Incentives and rewards
 - Norms (informal rules that govern behavior)
 - May need moderation to enforce formal rules
 - Social regulation (how do you get users and keep them?)
 - Social facilitation- to solve cold start problem, make it seem like there's others/it's active and you'll want to join too
 - Want to be able to scale to lots of users

4/8/25 was a bakeoff, 4/10/25 was a guest lecture.

4/15/25

- Conventional learning < Mastery learning < Tutoring
- Mastery learning is a feedback-corrective procedure where students solve problems while being taught and the next steps in their learning depend on their problem performance

- Cognitive tutors use cognitive models. Can be twice as effective as traditional learning, but getting a cognitive model is hard
- Want to use AI to help teachers and learners
- ReadingQuizMaker
 - Rather than having AI generate the questions based on a prompt and that's it, teachers are involved in the process and have more control over what content is used
 - Teachers can select text from the article, and have the AI transform it by paraphrasing, negating, or summarizing it. Then, the teacher can manually modify it.
 - Much easier for teachers to know what part of the text the question comes from and verify its accuracy
- Note co-pilot
 - 3 variations: automated AI (structured note block every 2 minutes), intermediate AI (summarize each sentence), minimal AI (real-time transcript). User can also add their own notes
 - Intermediate > minimal > automated, maybe because they had to interact by dragging and dropping blocks into their notes, but people prefer the automated the best
 - More assistance isn't always better!